

System Architecture

Microservices architecture, service responsibilities, request flow, and data ownership

Document status: Active

Version: 1.0

Last updated: 2026-09-26

1. Overview

FleetFlow is a modular fleet and logistics management platform designed around a microservices architecture. The platform provides role-based access for administrators, managers, drivers, and customers and is being developed as a production-oriented portfolio project.

The current frontend is a Next.js application and the backend is being split into independently deployable services. The first implemented backend service is the authentication service.

2. Architecture Goals

- Separate business domains into independently maintainable services.
- Keep authentication and authorization concerns centralized.
- Allow individual services to evolve without coupling the entire backend.
- Provide a responsive web application for operational users.
- Support role-based workflows for Admin, Manager, Driver, and Customer.
- Make the system deployable as separate frontend and backend services.
- Keep the architecture extensible for notifications, reporting, maintenance, and other future domains.

3. Repository Structure

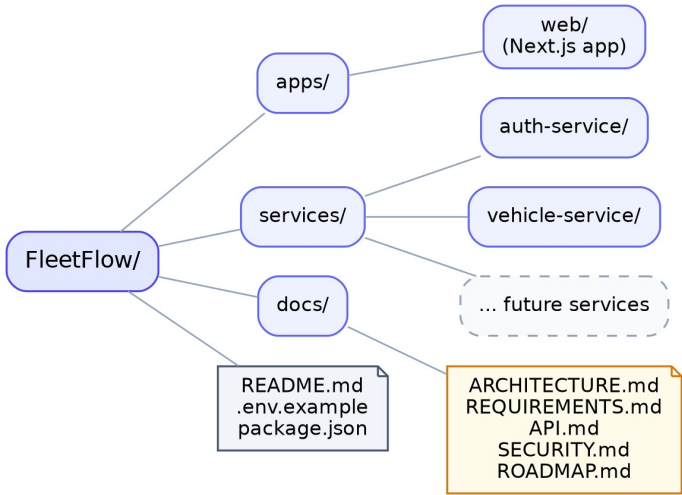


Figure 1 — FleetFlow repository structure

```
FleetFlow/
├── apps/
│   └── web/ # Next.js web application
├── services/
│   ├── auth-service/ # Authentication and user identity
│   ├── vehicle-service/ # Vehicle management
│   └── ... # Future domain services
├── docs/
│   ├── ARCHITECTURE.md
│   ├── REQUIREMENTS.md
│   ├── API.md
│   ├── SECURITY.md
│   └── ROADMAP.md
├── README.md
├── .env.example
└── package.json
```

4. Current Services

Web Application

Technology: Next.js 16.3.6, React, TypeScript, Tailwind CSS

Responsibilities:

- Authentication UI
- Login and registration
- Dashboard UI
- Vehicle management UI
- Vehicle request UI
- Vehicle service/history UI
- Role-aware navigation
- Theme switching
- Responsive presentation
- API communication with backend services

Auth Service

Technology: Fastify, TypeScript, MongoDB, Zod, JWT

Responsibilities:

- User registration
- User login
- Password authentication
- JWT issuance
- User identity
- Role information
- Authentication-related validation

Current roles:

```
admin
manager
driver
customer
```

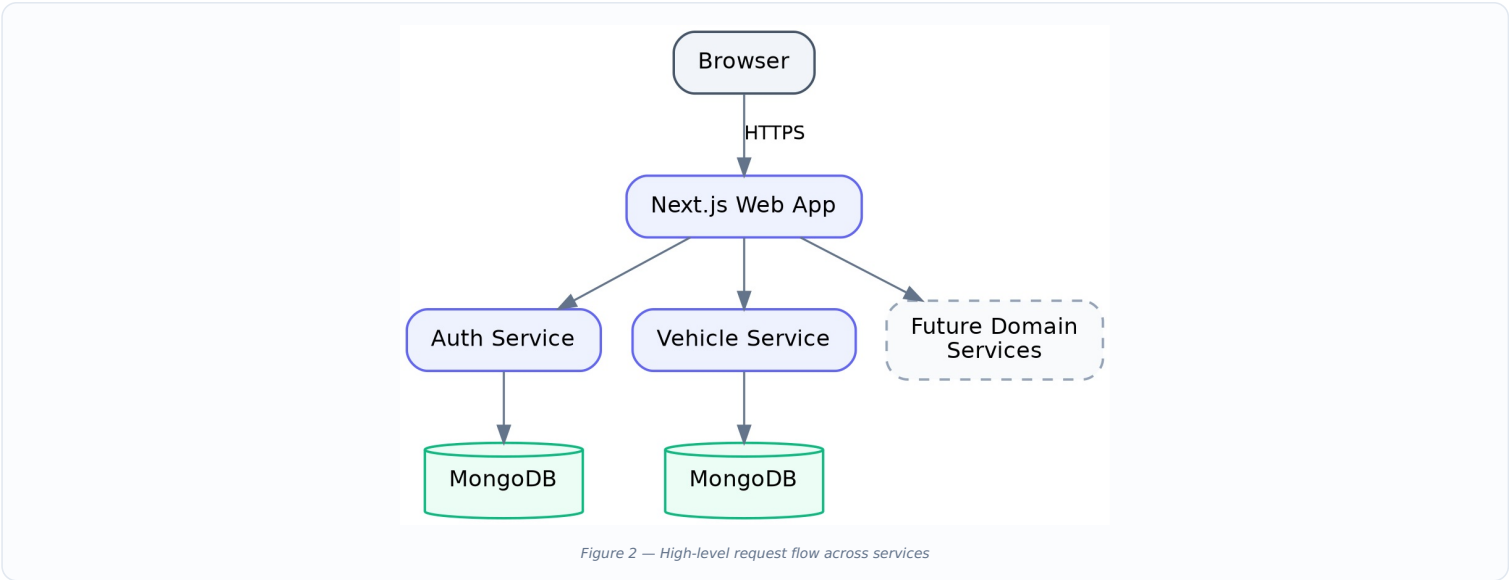
Vehicle Service

The vehicle domain is implemented as a separate backend concern and supports vehicle CRUD and status management.

Current domain responsibilities include:

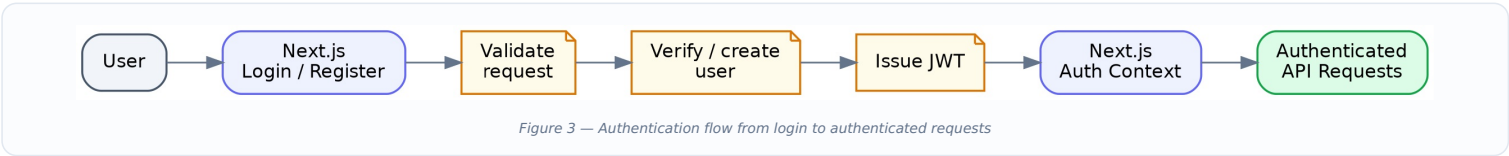
- Create vehicle
- List vehicles
- Retrieve a vehicle
- Update vehicle
- Deactivate vehicle
- Update vehicle status
- Vehicle-related validation

5. High-Level Request Flow



The web application should communicate with domain services through HTTP APIs rather than directly accessing service databases.

6. Authentication Flow



The frontend maintains authenticated user state through the application authentication context and attaches authentication information to protected API requests.

7. Role Model

Role	Primary responsibility
Admin	Platform-wide administration and configuration
Manager	Fleet and operational management
Driver	Driver-specific operational workflows
Customer	Requesting/consuming fleet-related services

Detailed permissions are defined in REQUIREMENTS.md .

8. Data Ownership Principle

Each microservice should own its domain data.

For example:

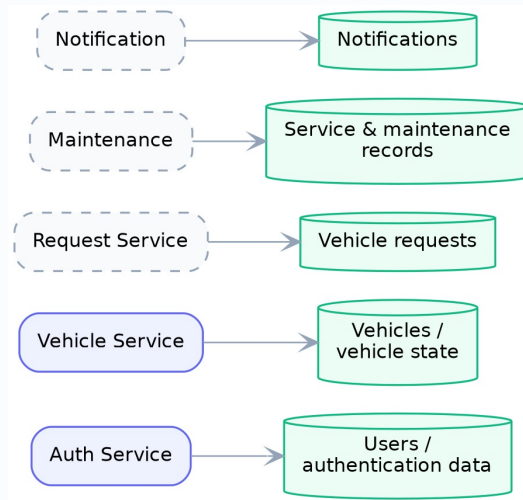


Figure 4 — Data ownership by service

Auth Service	→ Users / authentication data
Vehicle Service	→ Vehicles / vehicle state
Request Service	→ Vehicle requests
Maintenance	→ Service and maintenance records
Notification	→ Notifications

A service should not directly modify another service's database. Cross-domain operations should use APIs or an event-driven mechanism when introduced.

9. Communication Strategy

Current

- REST-style HTTP APIs
- JSON request/response payloads
- JWT-based authentication

Planned

- Service-to-service authentication
- Event-driven communication for selected workflows
- Centralized notification events
- Background jobs where asynchronous processing is useful

10. Frontend Architecture

The web application follows a component/context/API-client approach:

```
app/  
├─ pages/routes  
├─ components/  
├─ context/  
├─ lib/  
└─ styles
```

Important frontend concerns include:

- `AuthContext` for authentication state
- Shared API client
- Dashboard layouts
- Reusable UI components
- Role-aware rendering
- Light/dark theme support

11. Environment Configuration

Environment-specific values must be supplied through environment variables.

Examples:

```
JWT_SECRET  
MONGODB_URI  
NEXT_PUBLIC_API_URL  
AUTH_SERVICE_URL  
VEHICLE_SERVICE_URL
```

Actual secrets must never be committed to Git.

12. Scalability Principles

FleetFlow should remain scalable by:

1. Keeping services stateless where practical.
2. Separating domain ownership.
3. Avoiding direct cross-service database access.
4. Validating external input at service boundaries.
5. Keeping configuration outside source code.
6. Making services independently deployable.
7. Adding centralized logging and monitoring as deployment maturity increases.

13. Architecture Decisions

Why microservices?

FleetFlow is intentionally structured as a microservices project to demonstrate:

- Service boundaries
- Independent backend modules
- API contracts
- Authentication across services
- Deployment independence
- Production-oriented architecture

This also provides a realistic foundation for extending FleetFlow beyond a monolithic portfolio application.

14. Current vs Planned

Implemented / actively developed

- Next.js web application
- Authentication service
- JWT authentication foundation
- Role model
- Vehicle management
- Vehicle service/history UI
- Vehicle request workflow foundation
- Responsive dashboard architecture
- Light/dark theme

Planned / expanding

- Additional domain services
- Advanced vehicle assignment
- Driver management
- Maintenance workflows
- Notifications
- Reporting/analytics
- Automated testing
- Production deployment
- Observability